

(önceki sayfadan devam)

```
my_signal.connect(backend.my_handler) # Farklı bir alıcı.

connect_signals() # Bu kodu tekrar çalıştırmak başka bir alıcı kaydeder.
```

Bir bound method'u alıcı olarak kullanırken, benzersiz bir `dispatch_uid` sağlayarak birden fazla kayıt önlenebilir. Bu tanımlayıcı genellikle bir dize olacaktır, ancak herhangi bir `hashable` nesne yeterli olacaktır. Alıcı, her benzersiz `dispatch_uid` değeri için sinyale yalnızca bir kez bağlanacaktır:

```
from django.core.signals import request_finished

request_finished.connect(my_callback, dispatch_uid="my_unique_identifier")
```

3.23.2 Sinyalleri tanımlama ve gönderme

Uygulamalarınız sinyal altyapısından yararlanabilir ve kendi sinyallerini sağlayabilir.

Özel sinyaller ne zaman kullanılır

Sinyaller, hata ayıklamayı (debugging) zorlaştıran örtük (implicit) fonksiyon çağrılarıdır. Özel sinyalinizin (custom signal) göndericisi ve alıcısı projenizin içindeyse, açık (explicit) bir fonksiyon çağrısı kullanmanız daha iyi olur.

Sinyalleri tanımlama

```
class Signal
```

Tüm sinyaller `django.dispatch.Signal` örnekleridir (instances).

Örneğin:

```
import django.dispatch

pizza_done = django.dispatch.Signal()
```

Bu, bir `pizza_done` sinyali bildirir.

Sinyalleri gönderme

Django'da sinyalleri eşzamanlı (synchronously) olarak göndermenin iki yolu vardır.

```
Signal.send(sender, **kwargs)
```

```
Signal.send_robust(sender, **kwargs)
```

Sinyaller asenkron (asynchronously) olarak da gönderilebilir.

```
Signal.asend(sender, **kwargs)
```

```
Signal.asend_robust(sender, **kwargs)
```

Bir sinyal göndermek için, `Signal.send()`, `Signal.send_robust()`, `await Signal.asend()` veya `await Signal.asend_robust()` yöntemlerinden birini çağırın. `sender` argümanını (genellikle bir sınıftır) sağlamanız gerekir ve istediğiniz kadar anahtar kelime argümanı sağlayabilirsiniz.

Örneğin, `pizza_done` sinyelimizi göndermek şöyle görünebilir:

```
class PizzaStore:
    ...

    def send_pizza(self, toppings, size):
        pizza_done.send(sender=self.__class__, toppings=toppings, size=size)
    ...
```

Dört yöntemin tamamı, çağrılan alıcı fonksiyonların listesini ve yanıt değerlerini temsil eden bir tuple çiftleri listesi [(receiver, response), ...] döndürür.

`send()`, alıcı fonksiyonlar tarafından yükseltile istisnaların nasıl işlendiği konusunda `send_robust()`'tan farklıdır. `send()`, alıcılar tarafından yükseltile hiçbir istisnayı yakalamaz; sadece hataların yayılmasına izin verir. Böylece, hata anında tüm alıcılar bir sinyal hakkında bilgilendirilmeyebilir.

`send_robust()`, Python'un `Exception` sınıfından türetilen tüm hataları yakalar ve tüm alıcıların sinyalden haberdar edilmesini sağlar. Eğer bir hata oluşursa, hata örneği (error instance), hatayı yükselten alıcı için tuple çiftinde döndürülür.

İzlemeler (Tracebacks), `send_robust()` çağrılırken döndürülen hataların `__traceback__` özneliğinde bulunur.

`asend()`, `send()`'e benzer, ancak beklenmesi gereken (must be awaited) bir coroutine'dir:

```
async def asend_pizza(self, toppings, size):
    await pizza_done.asend(sender=self.__class__, toppings=toppings, size=size)
    ...
```

İster senkron (synchronous) ister asenkron (asynchronous) olsun, alıcılar kullanılan `send()` veya `asend()` yöntemine göre doğru şekilde uyarlanacaktır. Senkron alıcılar, `asend()` aracılığıyla çağrıldıklarında `sync_to_async()` kullanılarak çağrılacaktır. Asenkron alıcılar, `send()` aracılığıyla çağrıldıklarında `async_to_sync()` kullanılarak çağrılacaktır. Ara yazılım (middleware) durumuna benzer şekilde, alıcıları bu şekilde uyarlamanın küçük bir performans maliyeti vardır. Senkron/asenkron çağrı tarzı anahtarların sayısını azaltmak için, alıcıların çağrılmadan önce asenkron olup olmadıklarına göre gruplandırıldığını unutmayın. Bu, senkron bir alıcıdan sonra kaydedilen asenkron bir alıcının, senkron alıcıdan sonra yürütülebileceği anlamına gelir. Ek olarak, `async` alıcılar `asyncio.gather()` kullanılarak eşzamanlı olarak yürütülür.

Asenkron istek-yanıt döngüsündekiler hariç tüm yerleşik sinyaller `Signal.send()` kullanılarak gönderilir.

3.23.3 Sinyallerin bağlantısının kesilmesi

`Signal.disconnect(receiver=None, sender=None, dispatch_uid=None)`

Bir alıcıyı (receiver) bir sinyalden ayırmak için, `Signal.disconnect()` işlevini çağırın. Argümanlar `Signal.connect()` içinde açıklandığı gibidir. Eğer bir alıcının bağlantısı kesildiyse, metot `True` döndürür, aksi takdirde `False` döndürür. `sender`, `<app etiketi>.<model>`'e tembel (lazy) bir referans olarak iletildiğinde, bu metot her zaman `None` döndürür.

`receiver` argümanı, bağlantısı kesilecek kayıtlı alıcıyı (receiver) belirtir. Alıcıyı tanımlamak için `dispatch_uid` kullanılıyorsa, bu `None` olabilir.

3.24 Sistem kontrol çerçevesi

Sistem kontrol çerçevesi (system check framework), Django projelerini doğrulamak için tasarlanmış bir dizi statik kontroldür. Yaygın sorunları tespit eder ve bunların nasıl düzeltileceğine dair ipuçları (hints) sağlar. Çerçeve genişletilebilir, böylece kendi kontrollerinizi kolayca ekleyebilirsiniz.

Kontroller, `check` komutu aracılığıyla açıkça tetiklenebilir. Kontroller, `runserver` ve `migrate` dahil olmak üzere çoğu komuttan önce zımnın tetiklenir. Performans nedenleriyle, kontroller dağıtımda kullanılan WSGI yığınının (stack) bir parçası olarak çalıştırılmaz. Eğer dağıtım sunucunuzda sistem kontrollerini çalıştırmanız gerekiyorsa, bunları `check` kullanarak açıkça tetikleyin.

Ciddi hatalar, Django komutlarının (örneğin `runserver`) çalışmasını tamamen engelleyecektir. Küçük sorunlar konsola rapor edilir. Bir uyarının nedenini incelediyseniz ve onu görmezden gelmekten memnunsanız, proje ayarları dosyanızdaki `SILENCED_SYSTEM_CHECKS` belirli uyarıları gizleyebilirsiniz.

Django tarafından oluşturulabilecek tüm kontrollerin tam listesi [Sistem kontrol referansında](#) bulunabilir.

3.24.1 Kendi kontrollerinizi yazma

Çerçeve esnektir ve ihtiyacınız olabilecek her türlü başka kontrolü gerçekleştiren işlevler (functions) yazmanıza olanak tanır. Aşağıdaki, bir örnek taslak kontrol işlevidir:

```
from django.core.checks import Error, register

@register()
def example_check(app_configs, **kwargs):
    errors = []
    # ... your check logic here
    if check_failed:
        errors.append(
            Error(
                "bir hata",
                hint="Bir ipucu.",
```

(sonraki sayfada devam eder)